ASSIGNMENT – UNIT V

Manisha M
240701304

1. The Limits of Low-Code Platforms

Overview

Low-code platforms help people create applications using visual tools such as drag-and-drop components, forms, reports and workflows, with only a small amount of programming. They reduce development time and allow people with different levels of coding knowledge to build useful applications.

However, low-code platforms also have limitations. Advanced features may require custom code, and the platform may not provide complete control over design, performance or architecture. Organisations must also consider security, integration with other systems, licensing costs, scalability and vendor lock-in. Vendor lock-in means that moving an application and its data to another platform may be difficult or expensive. Low-code does not remove the need for testing, documentation or maintenance. Therefore, developers should select a platform based on the project's needs and understand when custom programming may be necessary.

20 Multiple-Choice Questions (MCQs)

1. What is a major benefit of low-code platforms?

A. They remove the need for testing
B. They speed up application development
C. They guarantee zero security risks
D. They work without requirements

Correct Answer: B — They speed up application development

Explanation: Low-code tools reduce the amount of manual coding and can speed up development.

2. Which is a common limitation of low-code platforms?

A. They cannot create forms
B. They always require machine code
C. They may restrict advanced customisation
D. They cannot store data

Correct Answer: C — They may restrict advanced customisation

Explanation: Some specialised features may be difficult to implement using built-in components.

3. What does vendor lock-in mean?

A. Being tied to a platform that is difficult to leave
B. Locking users out of an app
C. Encrypting source code
D. Preventing all updates

Correct Answer: A — Being tied to a platform that is difficult to leave

Explanation: Vendor lock-in can make migration to another platform costly or difficult.

4. Why might a complex application need pro-code development?

A. Pro-code avoids all documentation
B. Low-code has no visual tools
C. Complex requirements may exceed built-in features
D. Pro-code does not need testing

Correct Answer: C — Complex requirements may exceed built-in features

Explanation: Custom programming can support requirements that visual components cannot easily handle.

5. Which issue should be considered when selecting a low-code platform?

A. Only its logo
B. Security and data governance
C. Only the number of buttons
D. The developer's typing speed

Correct Answer: B — Security and data governance

Explanation: Applications must protect data and follow suitable access and governance rules.

6. What should be reassessed as a low-code application grows?

A. Only the logo
B. The number of headings
C. Performance and licensing needs
D. The developer's handwriting

Correct Answer: C — Performance and licensing needs

Explanation: Growth can affect platform limits, costs and application performance.

7. Which statement about low-code security is correct?

A. Security still requires configuration and review
B. Low-code apps cannot be attacked
C. Security is irrelevant when using templates
D. All platforms provide identical security

Correct Answer: A — Security still requires configuration and review

Explanation: Developers still need to configure permissions and check security.

8. What can make moving an app to another platform difficult?

A. The screen becomes larger
B. Platform-specific logic may need rewriting
C. Data is always transferred automatically
D. The app no longer needs users

Correct Answer: B — Platform-specific logic may need rewriting

Explanation: Different platforms may use different features, data models and scripting languages.

9. Which case may not suit standard low-code components alone?

A. A basic leave request form
B. A simple feedback survey

C. A routine approval workflow

D. A specialised algorithm with strict performance needs

Correct Answer: D — A specialised algorithm with strict performance needs

Explanation: A specialised algorithm may need greater control than built-in components provide.

10. Why estimate total cost of ownership?

A. To count only the first day's cost

B. To ignore subscription fees

C. To include licences, integration, maintenance and scaling

D. To avoid comparing platforms

Correct Answer: C — To include licences, integration, maintenance and scaling

Explanation: The full cost includes more than the initial setup.

11. What is shadow IT?

A. Approved software documented by IT

B. Unmanaged tools or apps used without proper IT oversight

C. A backup server

D. A user interface theme

Correct Answer: B — Unmanaged tools or apps used without proper IT oversight

Explanation: Shadow IT refers to technology used without the organisation's proper oversight.

12. Which practice helps control application sprawl?

A. Governance and an application inventory

B. Allowing every app unrestricted access

C. Avoiding documentation

D. Sharing administrator passwords

Correct Answer: A — Governance and an application inventory

Explanation: Governance helps organisations track applications and manage risks.

13. When might a built-in connector be insufficient?

A. The user can log in

B. A form has a title

C. The external service needs a custom integration

D. The app has a report

Correct Answer: C — The external service needs a custom integration

Explanation: A custom API or additional code may be needed for unsupported integrations.

14. What is a sensible response to platform limitations?

A. Ignore them

B. Assess requirements and extend with code when appropriate

C. Remove all validation

D. Skip user feedback

Correct Answer: B — Assess requirements and extend with code when appropriate

Explanation: The team should assess the limitation and choose a suitable technical solution.

15. Why is testing necessary in low-code projects?

A. Visual tools cannot contain mistakes
B. Testing is only for pro-code
C. Workflows, permissions and integrations can fail
D. Testing slows every app permanently

Correct Answer: C — Workflows, permissions and integrations can fail

Explanation: Visual workflows and integrations can still contain errors.

16. What may happen if a provider discontinues a service?

A. The app gains unlimited features
B. Support and migration may become difficult
C. The code becomes open source automatically
D. Data governance is no longer needed

Correct Answer: B — Support and migration may become difficult

Explanation: A discontinued service may require migration or changes to the application.

17. What should be checked before connecting sensitive data?

A. Access controls and data-handling policies
B. Font size only
C. Dashboard colour
D. Number of form fields only

Correct Answer: A — Access controls and data-handling policies

Explanation: Sensitive information requires appropriate access and handling controls.

18. What determines whether a low-code app can scale?

A. Every app scales infinitely
B. The platform, design and workload
C. Scaling never changes cost
D. The logo

Correct Answer: B — The platform, design and workload

Explanation: Scalability depends on the platform's limits and how the application is designed.

19. How does documentation help a low-code project?

A. It replaces security
B. It prevents all bugs
C. It records workflows, integrations and ownership
D. It makes testing unnecessary

Correct Answer: C — It records workflows, integrations and ownership

Explanation: Documentation helps people understand and maintain the application.

20. What is a sensible platform-selection approach?

A. Choose the first platform found
B. Choose only by popularity
C. Avoid a prototype
D. Compare requirements, limits, security, cost and portability

Correct Answer: D — Compare requirements, limits, security, cost and portability

Explanation: A platform should be selected based on the project's needs and long-term risks.

2. Glimpses of Pro Code in Zoho Creator and Catalyst

Overview

Pro-code development means writing source code to create custom features and control application behaviour. Zoho Creator combines visual application building with code-based extensions. Its Deluge scripting language is used to automate workflows, validate data, update records and connect application components. Depending on the feature, developers can also use functions and integrations to extend Creator applications.

Zoho Catalyst is a cloud development platform for building application backends and services. It supports server-side functions using supported programming environments, including Java, Node.js and Python in applicable Catalyst runtimes. Developers can use APIs, SDKs, testing, deployment and monitoring to build customised solutions. These platforms show that low-code and pro-code can work together: visual tools handle common tasks, while custom code is useful for specialised logic, integrations and advanced requirements.

20 Multiple-Choice Questions (MCQs)

1. What does pro-code development primarily involve?

A. Writing and managing source code
B. Using only paper forms
C. Avoiding all programming
D. Using only drag-and-drop controls

Correct Answer: A — Writing and managing source code

Explanation: Pro-code relies on programming to implement application behaviour.

2. Which scripting language is commonly used in Zoho Creator?

A. COBOL
B. Deluge
C. Swift
D. MATLAB

Correct Answer: B — Deluge

Explanation: Deluge is Zoho's scripting language used in Creator and other Zoho products.

3. What is a common use of a Deluge function?

A. Replacing the internet
B. Designing hardware
C. Reusing custom business logic

D. Formatting an operating system

Correct Answer: C — Reusing custom business logic

Explanation: Functions let developers reuse logic instead of repeating it.

4. Which task suits a Creator workflow function?

A. Repairing a server physically
B. Changing a monitor cable
C. Creating a programming language
D. Validating a form and updating related records

Correct Answer: D — Validating a form and updating related records

Explanation: Workflow code can validate entries and perform actions on records.

5. What is Zoho Catalyst designed to support?

A. Cloud-based application and backend development
B. Only offline word processing
C. Only image editing
D. Hardware manufacturing

Correct Answer: A — Cloud-based application and backend development

Explanation: Catalyst provides cloud services for building application functionality.

6. Which list includes programming environments supported by Catalyst functions?

A. HTML, CSS and SQL only
B. Java, Node.js and Python
C. Excel, Word and PowerPoint
D. Scratch, Logo and BASIC only

Correct Answer: B — Java, Node.js and Python

Explanation: Catalyst supports serverless function runtimes for supported languages, including these.

7. What does serverless computing mean?

A. No physical servers exist
B. Developers manage every server manually
C. The provider manages much of the underlying infrastructure
D. Applications cannot use a backend

Correct Answer: C — The provider manages much of the underlying infrastructure

Explanation: Serverless abstracts much of the server management from developers.

8. What is an API commonly used for?

A. Increasing screen brightness
B. Replacing all databases
C. Writing documentation only
D. Enabling software systems to communicate

Correct Answer: D — Enabling software systems to communicate

Explanation: APIs define ways for software systems to exchange data or request actions.

9. Why are reusable functions valuable?

A. They reduce repeated code and improve maintainability
B. They guarantee no bugs
C. They remove the need for input
D. They prevent integration

Correct Answer: A — They reduce repeated code and improve maintainability

Explanation: Reusable functions make logic easier to maintain and update.

10. A custom API can allow selected application logic to be used by:

A. Only a printer
B. External applications or services
C. Only a keyboard
D. No other system

Correct Answer: B — External applications or services

Explanation: APIs can expose selected functionality to other authorised systems.

11. Which is an example of a pro-code task?

A. Selecting a theme only
B. Dragging a text field only
C. Writing a custom algorithm in a function
D. Choosing a form title

Correct Answer: C — Writing a custom algorithm in a function

Explanation: Writing a custom algorithm involves programming.

12. What is the purpose of an SDK?

A. Replace source code with a picture
B. Store passwords in plain text
C. Stop APIs from working
D. Provide tools and libraries for developing with a platform

Correct Answer: D — Provide tools and libraries for developing with a platform

Explanation: SDKs provide useful libraries, tools and guidance for developers.

13. Which approach best combines visual and code capabilities?

A. Build common forms visually and add code for specialised logic
B. Avoid all workflows
C. Write every feature from scratch
D. Disable validation

Correct Answer: A — Build common forms visually and add code for specialised logic

Explanation: A mixed approach uses visual development for standard tasks and code where needed.

14. What should a developer do before deploying a custom function?

A. Assume it works
B. Test normal, invalid and failure cases

C. Remove error handling

D. Share credentials publicly

Correct Answer: B — Test normal, invalid and failure cases

Explanation: Testing helps identify errors before users depend on the function.

15. What is the role of authentication in an API?

A. Change the font

B. Increase image resolution

C. Help verify identity or access rights

D. Replace data validation

Correct Answer: C — Help verify identity or access rights

Explanation: Authentication checks identity; authorisation determines permitted actions.

16. Which is an example of integration logic?

A. Changing a background

B. Renaming a local folder

C. Printing a blank sheet

D. Sending data to an external service through an API

Correct Answer: D — Sending data to an external service through an API

Explanation: Integration logic enables different applications to exchange information.

17. What does a backend function usually handle?

A. Server-side business logic or data processing

B. Monitor settings only

C. Keyboard hardware design

D. Physical network cabling

Correct Answer: A — Server-side business logic or data processing

Explanation: Backend functions process data and implement server-side behaviour.

18. Why might a team use Catalyst for a custom backend?

A. It removes the need for any code

B. It provides cloud services for functions and application backends

C. It works only as a spreadsheet

D. It cannot connect to a client app

Correct Answer: B — It provides cloud services for functions and application backends

Explanation: Catalyst offers managed cloud services that support backend development.

19. Which practice improves code maintainability?

A. Putting all logic in one unlabelled block

B. Avoiding all comments

C. Using meaningful names and modular functions

D. Duplicating code everywhere

Correct Answer: C — Using meaningful names and modular functions

Explanation: Clear names and modular code make changes easier to understand.

20. What is the key lesson from Creator and Catalyst?

A. Visual tools make programming obsolete
B. Pro-code and low-code cannot coexist
C. Every app must be written from scratch
D. Visual development can be extended with code for greater control

Correct Answer: D — Visual development can be extended with code for greater control

Explanation: Both approaches can be combined to meet different application needs.

3. The No-Code to Low-Code to Pro-Code Spectrum Revisited

Overview

No-code, low-code and pro-code are points on a development spectrum. No-code uses visual builders, templates and configuration with little or no hand-written code. It is useful for simple applications, forms and prototypes. Low-code also uses visual tools but allows scripting, APIs and custom code when needed. Pro-code relies more directly on programming languages, frameworks and developer-controlled architecture.

These approaches are not completely separate. A project may begin as a no-code prototype, develop into a low-code application with automated workflows, and later use pro-code services for complex requirements. The best choice depends on application complexity, customisation, performance, security, team skills and maintenance. The goal is not to use the most code possible; it is to select the approach that solves the problem effectively and responsibly.

20 Multiple-Choice Questions (MCQs)

1. Which approach relies most heavily on visual configuration with little or no coding?

A. Pro-code
B. No-code
C. Assembly language
D. Manual server administration

Correct Answer: B — No-code

Explanation: No-code tools are designed to build applications mainly through visual controls.

2. Which approach commonly combines visual tools with optional scripting?

A. No-code only
B. Hardware coding
C. Low-code
D. Paper-based development

Correct Answer: C — Low-code

Explanation: Low-code provides visual development plus opportunities to add code.

3. Which approach generally offers the greatest direct control over code and architecture?

A. Pro-code

B. No-code
C. Template-only design
D. Spreadsheet formatting

Correct Answer: A — Pro-code

Explanation: Pro-code gives developers more direct control over implementation.

4. A teacher needs a simple registration form without programming. What is a sensible starting point?

A. Build an operating system
B. Write a compiler
C. Create a database engine
D. Use a no-code form builder

Correct Answer: D — Use a no-code form builder

Explanation: A no-code form builder is suitable for a straightforward form.

5. A team needs visual forms plus custom API logic. Which approach may fit?

A. No-code only
B. Low-code
C. No software platform
D. Manual data entry

Correct Answer: B — Low-code

Explanation: Low-code supports visual components and custom integrations.

6. Why is the spectrum flexible?

A. Every project must use all three
B. Coding skill never matters
C. Teams can combine approaches based on requirements
D. No-code always replaces pro-code

Correct Answer: C — Teams can combine approaches based on requirements

Explanation: Projects can use different methods for different parts of the solution.

7. Which factor should guide the choice of approach?

A. Requirements and complexity
B. The longest possible code
C. Logo colours
D. Developer preference alone

Correct Answer: A — Requirements and complexity

Explanation: The solution should match what the application must do.

8. What is a common strength of no-code tools?

A. Unlimited algorithmic control
B. Automatic elimination of all risks
C. Guaranteed portability
D. Fast creation of standard apps by non-specialists

Correct Answer: D — Fast creation of standard apps by non-specialists

Explanation: Visual tools help people create simple apps without extensive coding.

9. What is a possible low-code advantage over no-code?

A. It forbids integrations
B. It allows custom logic through code or scripting
C. It cannot build workflows
D. It removes user testing

Correct Answer: B — It allows custom logic through code or scripting

Explanation: Low-code gives developers options for adding custom logic.

10. What is a possible pro-code advantage for specialised systems?

A. No maintenance is required
B. No programming knowledge is needed
C. Greater control over implementation and architecture
D. No security testing is needed

Correct Answer: C — Greater control over implementation and architecture

Explanation: Pro-code can provide fine-grained control for specialised requirements.

11. Which statement is most accurate?

A. The categories can overlap in real projects
B. No-code apps cannot be useful
C. Pro-code always costs less
D. Low-code cannot use APIs

Correct Answer: A — The categories can overlap in real projects

Explanation: Teams can combine tools and approaches within one solution.

12. A prototype must be built quickly to test an idea. What may be appropriate?

A. Only a large custom backend
B. Rewriting a database engine
C. Avoiding feedback
D. A no-code or low-code prototype

Correct Answer: D — A no-code or low-code prototype

Explanation: A visual prototype can help test an idea quickly with users.

13. When might a team move from low-code toward pro-code?

A. When a form needs a title
B. When advanced requirements exceed platform capabilities
C. When a user changes a password
D. When a report has a heading

Correct Answer: B — When advanced requirements exceed platform capabilities

Explanation: Complex needs may require more direct programming control.

14. Which role can connect business needs with low-code implementation?

A. Hardware technician only
B. Graphic artist only
C. Citizen developer or business analyst
D. End user with no project context
Correct Answer: C — Citizen developer or business analyst

Explanation: These roles help translate business needs into workable applications.

15. What is citizen development?

A. Business users creating applications with approved tools
B. Developing city roads
C. Coding only in assembly language
D. Building apps without requirements
Correct Answer: A — Business users creating applications with approved tools

Explanation: Citizen developers use accessible tools to build solutions within organisational rules.

16. What is a risk of treating no-code as a complete replacement for engineering?

A. Forms become impossible
B. Every project gets faster forever
C. APIs disappear
D. Complex security, integration and maintenance needs may be overlooked
Correct Answer: D — Complex security, integration and maintenance needs may be overlooked

Explanation: Some applications still need professional engineering and review.

17. What does abstraction mean in visual development?

A. Removing all application behaviour
B. Hiding technical details behind simpler controls
C. Deleting the interface
D. Preventing data storage
Correct Answer: B — Hiding technical details behind simpler controls

Explanation: Abstraction makes technical operations easier to use through simplified tools.

18. Which project likely needs strong pro-code involvement?

A. A basic survey
B. A simple event sign-up page
C. A high-performance service with specialised algorithms
D. A standard contact form
Correct Answer: C — A high-performance service with specialised algorithms

Explanation: Specialised algorithms and performance needs may require custom engineering.

19. How can an organisation use the spectrum responsibly?

A. Define standards, review risks and select tools for each use case
B. Let every app publish without review

C. Never document apps

D. Avoid professional developers

Correct Answer: A — Define standards, review risks and select tools for each use case

Explanation: Standards and review help maintain security and quality.

20. What is the main idea of revisiting the spectrum?

A. Only no-code matters

B. Low-code is always best

C. Pro-code is outdated

D. Choose a balanced approach based on the problem

Correct Answer: D — Choose a balanced approach based on the problem

Explanation: The best approach depends on the application and its requirements.

4. Career Pathways

Overview

Low-code and pro-code development offer several career opportunities. Beginners can start by learning application configuration, form design, workflow creation and basic data modelling. With practice, they may become low-code developers, business application developers, automation specialists or platform administrators.

Learning programming, APIs, databases, authentication, testing and deployment can open pathways to pro-code roles such as software developer, backend developer, cloud developer or integration engineer. Other possible roles include business analyst, solution architect, QA engineer, security specialist and technical consultant. Employers also value communication, teamwork, debugging, documentation and problem-solving. Students can prepare by building small projects, learning one platform well, practising programming fundamentals and maintaining a portfolio that explains the problem, solution, tools used and their personal contribution.

20 Multiple-Choice Questions (MCQs)

1. Which is a suitable beginner task in low-code development?

A. Designing a simple form and workflow

B. Building a CPU

C. Writing an operating system kernel

D. Designing a new programming language

Correct Answer: A — Designing a simple form and workflow

Explanation: A simple form and workflow help beginners learn application development.

2. What does a business analyst commonly do?

A. Repair laptops only

B. Converts business needs into clear requirements

C. Avoid stakeholder discussions

D. Manage no project information

Correct Answer: B — Converts business needs into clear requirements

Explanation: Business analysts understand needs and communicate them to the development team.

3. Which skill is useful for both low-code and pro-code careers?

A. Memorising logos
B. Avoiding teamwork
C. Problem-solving
D. Ignoring user needs

Correct Answer: C — Problem-solving

Explanation: Problem-solving is essential when designing and debugging applications.

4. What does a QA engineer focus on?

A. Selling licences only
B. Designing office furniture
C. Writing slogans only
D. Testing software quality and identifying defects

Correct Answer: D — Testing software quality and identifying defects

Explanation: QA engineers check whether software works as expected.

5. Which skill helps a developer connect two applications?

A. API and integration knowledge
B. Handwriting speed
C. Screen brightness control
D. Image cropping only

Correct Answer: A — API and integration knowledge

Explanation: APIs allow applications to communicate and exchange data.

6. What can a portfolio demonstrate to employers?

A. Only attendance
B. Practical projects and the candidate's contribution
C. Guaranteed employment
D. Personal passwords

Correct Answer: B — Practical projects and the candidate's contribution

Explanation: A portfolio provides evidence of practical skills and project experience.

7. Which career may focus on designing the overall technical solution?

A. Data entry operator only
B. End user
C. Solution architect
D. Typist

Correct Answer: C — Solution architect

Explanation: Solution architects design how components work together to meet requirements.

8. Why should a low-code developer learn programming fundamentals?

A. To avoid all tools
B. To stop using logic

C. To remove validation

D. To understand custom logic and extend platform capabilities

Correct Answer: D — To understand custom logic and extend platform capabilities

Explanation: Programming knowledge helps developers solve problems beyond built-in features.

9. Which is an important professional habit?

A. Documenting changes and decisions

B. Hiding defects

C. Sharing credentials

D. Skipping reviews

Correct Answer: A — Documenting changes and decisions

Explanation: Documentation supports teamwork and future maintenance.

10. Which technical area is valuable for application developers?

A. Unrelated trivia only

B. Databases and data modelling

C. Avoiding data structures

D. Never learning APIs

Correct Answer: B — Databases and data modelling

Explanation: Most applications need structured data and a clear data model.

11. What is a good way to improve debugging skills?

A. Guess without checking

B. Delete every error message

C. Reproduce the issue and test possible causes systematically

D. Stop testing after the first run

Correct Answer: C — Reproduce the issue and test possible causes systematically

Explanation: A systematic approach helps identify the real cause of a bug.

12. Which role commonly manages platform configuration and access?

A. Film critic

B. Copy editor

C. Photographer only

D. Platform administrator

Correct Answer: D — Platform administrator

Explanation: Platform administrators manage configuration, users and permissions.

13. Why are communication skills important in software projects?

A. Developers must clarify needs and explain technical choices

B. Communication replaces all coding

C. It removes testing

D. It guarantees no changes

Correct Answer: A — Developers must clarify needs and explain technical choices

Explanation: Clear communication helps teams build the right solution.

14. Which learning sequence is sensible for a beginner?

A. Start only with advanced distributed systems
B. Build simple apps, learn logic, then practise APIs and custom code
C. Avoid small projects
D. Learn no fundamentals

Correct Answer: B — Build simple apps, learn logic, then practise APIs and custom code

Explanation: Gradually increasing project difficulty helps build confidence and skills.

15. Which is an example of an automation specialist's work?

A. Repairing a road
B. Designing a movie poster only
C. Automating repetitive business processes
D. Avoiding workflow rules

Correct Answer: C — Automating repetitive business processes

Explanation: Automation specialists reduce manual work by designing workflows.

16. What should a project description in a portfolio include?

A. Passwords and private keys
B. Only the project name
C. Unverified claims
D. The problem, solution, tools used and personal contribution

Correct Answer: D — The problem, solution, tools used and personal contribution

Explanation: This information helps employers understand the project and the candidate's role.

17. Why is continuous learning important?

A. Platforms, APIs and development practices change
B. Skills never change
C. Documentation becomes useless
D. Testing tools disappear

Correct Answer: A — Platforms, APIs and development practices change

Explanation: Developers need to keep up with evolving platforms and practices.

18. Which pro-code pathway may follow low-code experience?

A. No technical role is possible
B. Backend or cloud application developer
C. Only data entry
D. Only hardware sales

Correct Answer: B — Backend or cloud application developer

Explanation: Learning code and cloud concepts can open backend and cloud development roles.

19. What does a security specialist help with?

A. Choosing decorative fonts only
B. Avoiding access control
C. Reviewing threats, permissions and secure practices
D. Removing audit logs

Correct Answer: C — Reviewing threats, permissions and secure practices

Explanation: Security specialists help protect applications, users and data.

20. What is useful when choosing a career pathway?

A. Follow trends without interest
B. Avoid practice
C. Select only by job title
D. Match interests with skills, practice and job requirements

Correct Answer: D — Match interests with skills, practice and job requirements

Explanation: A good pathway matches personal interests with the skills employers need.

5. The Future of Low-Code

Overview

The future of low-code is likely to include closer integration with artificial intelligence, automation, cloud services and pro-code development. AI-assisted tools may help users create forms, suggest workflows, explain errors or generate starter code from natural-language instructions. Platforms may also improve reusable components, API connectivity, governance, monitoring and deployment.

These developments can make software creation faster, but they do not remove the need for human judgement. Generated applications must still be checked for correctness, security, privacy, accessibility, performance and compliance. Organisations will need clear ownership and policies for data and AI use. Low-code is therefore more likely to complement professional software engineering than completely replace it. Its long-term value will depend on balancing speed and accessibility with reliability, maintainability and responsible governance.

20 Multiple-Choice Questions (MCQs)

1. Which technology is likely to influence the future of low-code?

A. Artificial intelligence
B. Typewriters only
C. Paper filing alone
D. Floppy disks only

Correct Answer: A — Artificial intelligence

Explanation: AI can assist with creating and improving application components.

2. How might AI assist a low-code developer?

A. Guarantee every output is correct
B. Suggest workflows or generate starter code
C. Remove the need for requirements

D. Make security irrelevant

Correct Answer: B — Suggest workflows or generate starter code

Explanation: AI can provide suggestions and drafts that developers review.

3. Why must AI-generated code be reviewed?

A. It is always unusable
B. It cannot contain logic
C. It may contain errors or security weaknesses
D. It never runs

Correct Answer: C — It may contain errors or security weaknesses

Explanation: Generated code can be incorrect or unsafe, so testing and review remain necessary.

4. Which trend can help low-code applications connect with other systems?

A. Removing all interfaces
B. Avoiding networks
C. Using only paper records
D. Improved API and integration support

Correct Answer: D — Improved API and integration support

Explanation: APIs and connectors make it easier to exchange data between systems.

5. What is a likely role for low-code in future organisations?

A. Complementing professional software development
B. Replacing every programmer immediately
C. Removing all IT governance
D. Eliminating testing

Correct Answer: A — Complementing professional software development

Explanation: Low-code can support routine tasks while engineers handle complex needs.

6. Why will governance remain important?

A. Every app is automatically approved
B. Applications still handle data, access and business processes
C. No-code apps have no users
D. Governance prevents all innovation

Correct Answer: B — Applications still handle data, access and business processes

Explanation: Governance helps control security, access, compliance and ownership.

7. Which practice supports responsible AI-assisted development?

A. Publishing generated code without review
B. Entering confidential data into any tool
C. Validating outputs and protecting sensitive information
D. Ignoring licences

Correct Answer: C — Validating outputs and protecting sensitive information

Explanation: Responsible development includes review and protection of private data.

8. What does human-in-the-loop mean?

A. Humans are removed from decisions
B. A system only runs offline
C. No one reviews the result
D. People review or guide important automated decisions

Correct Answer: D — People review or guide important automated decisions

Explanation: People remain involved in checking or guiding important outcomes.

9. Which capability may become more common in low-code platforms?

A. AI-assisted debugging and explanations
B. Guaranteed bug-free apps
C. No access controls
D. No version history

Correct Answer: A — AI-assisted debugging and explanations

Explanation: AI tools may help developers understand errors and suggest fixes.

10. Why will professional developers still be needed?

A. Visual tools cannot create any app
B. Complex architecture, security and specialised logic need engineering judgement
C. Coding is prohibited
D. APIs will no longer exist

Correct Answer: B — Complex architecture, security and specialised logic need engineering judgement

Explanation: Professional engineering is still important for complex and high-risk systems.

11. Which is a realistic expectation about automation?

A. It removes all human responsibility
B. It never needs monitoring
C. It can reduce repetitive work but still needs oversight
D. It guarantees every workflow is fair

Correct Answer: C — It can reduce repetitive work but still needs oversight

Explanation: Automation can save time but must be monitored and evaluated.

12. What is one future challenge for low-code platforms?

A. Too few buttons
B. No forms can be built
C. Everyone stops using computers
D. Maintaining security and governance as adoption grows

Correct Answer: D — Maintaining security and governance as adoption grows

Explanation: More applications can mean more access, security and compliance risks.

13. How can reusable components help development?

A. Reduce duplicated effort and encourage consistency

B. Prevent every integration
C. Eliminate requirements
D. Make apps impossible to maintain

Correct Answer: A — Reduce duplicated effort and encourage consistency

Explanation: Reusable components can make common features faster to build consistently.

14. What should be considered before adopting an AI feature?

A. Only its name
B. Accuracy, privacy, security and suitability
C. Only its icon colour
D. Whether it removes human review

Correct Answer: B — Accuracy, privacy, security and suitability

Explanation: AI features should be assessed for quality and risk before use.

15. Which statement about low-code and jobs is most balanced?

A. All IT jobs will disappear
B. No new skills are needed
C. Roles may change, increasing the value of platform, integration and engineering skills
D. Developers will never use visual tools

Correct Answer: C — Roles may change, increasing the value of platform, integration and engineering skills

Explanation: The work may change, and a combination of platform and engineering skills will remain valuable.

16. What does scalability mean for an application?

A. Changing a logo
B. Making text bold
C. Adding a form title
D. Handling growth in users, data or workload appropriately

Correct Answer: D — Handling growth in users, data or workload appropriately

Explanation: A scalable application can handle increased demand within its design limits.

17. What can improve trust in AI-generated application logic?

A. Testing, review, auditability and clear ownership
B. Hiding the code
C. Disabling logs
D. Skipping validation

Correct Answer: A — Testing, review, auditability and clear ownership

Explanation: Review and traceability help teams assess whether generated logic is reliable.

18. Why may low-code support digital transformation?

A. It prevents all business change
B. It helps digitise workflows and build apps more quickly

C. It works only for entertainment

D. It removes the need to understand processes

Correct Answer: B — It helps digitise workflows and build apps more quickly

Explanation: Low-code can help organisations replace manual processes with digital applications.

19. What is a benefit of better cloud integration?

A. Applications can never fail

B. Every service becomes free

C. Apps can use managed services without building every component from scratch

D. Data no longer needs protection

Correct Answer: C — Apps can use managed services without building every component from scratch

Explanation: Managed cloud services can provide capabilities without requiring teams to build everything themselves.

20. What will help determine the long-term success of low-code?

A. Speed alone

B. AI alone

C. The number of templates alone

D. A balance of productivity, reliability, security and maintainability

Correct Answer: D — A balance of productivity, reliability, security and maintainability

Explanation: Long-term success requires more than fast development; apps must remain safe and maintainable.